

HXC-159 06_risk_register

HXC-159 — Advance Risk Register

Table of contents

1. How to read this register

2. Critical risks

R-19 — Executing the task as literally worded would make both agents measurably WORSE

R-01 — The task's premise does not hold: HelixSkills ships nothing to incorporate

R-02 — Nested own-org submodule chain: adding skills violates CONST-051(C)

R-03 — Live 46-commit constitution pin skew, with measurable capability loss

3. High risks

R-04 — Three disjoint skills corpora with zero reconciliation and zero name overlap

R-05 — Skill filename case is split; one skill has no document at all

R-06 — A tracked, already-broken skill symlink points at a foreign-OS path

R-07 — Cross-consumer capability parity has no enforcing mechanism

R-08 — Test-surface explosion makes §11.4.169 unaffordable if scoped naively

4. Medium risks

R-09 — Host lacks X11/GL headers; any GUI-adjacent skill surface cannot build here

R-10 — §11.4.228's mandated extensions catalogue does not exist in HelixCode

R-11 — Governance load per new submodule is real and easy to half-do

R-12 — `.mcp.json` hard-codes constitution skill paths, re-coupling the consumer

R-13 — Stale branches in the source repo invite integrating the wrong ref

R-14 — Skill content is executable instruction: a malicious or buggy skill is a live attack surface

- 5. Bridge-integration risks (added Rev 2) and resolved items
 - R-20 — The middle of the Speckit ↔ Superpowers Bridge is designed but NOT installed
 - R-21 — The Bridge's execution layer DOES NOT EXIST, and two of the three remaining layers are single-maintainer hobby projects
 - R-22b — Namespace collision is narrower than feared, but the biting case is undocumented
 - R-22 — .claude/skills/ namespace collision — now REAL, and silent
 - R-23 — A third constitution now exists in this repo (CONST-059 canonical-root ambiguity)
 - R-24 — §11.4.164 auto-propagation is only half-wired: the consumer registrars do not exist
 - R-25 — replace directives are ignored outside the main module (10 already in play)
 - R-26 — Root and inner modules declare the SAME module path
 - R-27 — /speckit.taskstoissues forks the workable-items single source of truth
 - R-28 — Corpus 2 is not reusable as-is (formerly R-18)
- Resolved from Rev 1
- 6. Summary table
- 7. Honest boundary

HXC-159 — Advance Risk Register

Field	Value
Document	06_risk_register.md
Work item	HXC-159 (Task, In-progress, High)
Phase	1b — advance risk detection (BEFORE implementation)
Revision	2
Last modified	2026-07-29
Author	(T1/main - claude1 - opus - xhigh)
Status summary	29 risks enumerated. 27 EVIDENCED (measured this session), 2 ANTICIPATED. 5 rated CRITICAL. Each carries a designed mitigation + proof plan. Rev 2 folds in the complete §11.4.150 research return (04_deep_research.md) and the mid-flight Bridge requirement.

Field	Value
Anchors	Operator mandate (gaps detected <i>in advance</i> , tackled with rock-solid risk-free solutions) · §11.4.6 · §11.4.28(C)/CONST-051(C) · §11.4.169 · §11.4.157 · §11.4.31/CONST-054 · §11.4.36/CONST-056 · §11.4.30 · §11.4.77 · §11.4.164 · §11.4.228 · CONST-059

Revision 2 changes. Added R-19 (skill shadowing — now the single most dangerous risk) and R-20...R-28 (Bridge layers, namespace collision, third constitution, replace semantics, duplicate module path, registrar gap, tracker fork). Upgraded R-05 and R-14 from partial to fully EVIDENCED. Resolved R-15/R-16/R-17 from ANTICIPATED using cited research.

Table of contents

1. [How to read this register](#)
2. [Critical risks](#)
3. [High risks](#)
4. [Medium risks](#)
5. [Anticipated risks pending research](#)
6. [Summary table](#)
7. [Honest boundary](#)

1. How to read this register

The operator's mandate is explicit that this is a **planning** deliverable:

"Any gaps, shortcomings, weak spots, danger zones, issues, or any inconsistencies MUST BE detected in advance during the planning and implementation process and properly tackled with comprehensive rock-solid solutions implementations, risk-free and safe!"

So every row carries four fields, and a row without a designed mitigation is not a row — it is an unfinished one.

Each risk is labelled by evidence class, per §11.4.6:

- **EVIDENCED** — measured in this session; the command and its output are shown. A fact.

- **ANTICIPATED** — a structural consequence reasoned from evidenced facts, or a risk whose confirming research is still in flight. Explicitly *not* a fact.

Nothing here is labelled from memory or assumption.

2. Critical risks

R-19 — Executing the task as literally worded would make both agents measurably WORSE

Class: EVIDENCED (peer-reviewed measurement + measured local corpus size) · **Severity:** CRITICAL

What it is. This is the §11.4.150(C) "bigger underlying problem", and it is the most dangerous item in this register because it is **invisible to every test that would be written for HXC-159**.

Databricks, *More Skills, Worse Agents? Skill Shadowing Degrades Performance When Expanding Skill Libraries* (arXiv:2605.24050v1, 2026-05-21) — SkillsBench, 38 oracle (task, model) pairs, 2,545 trajectories, Claude Haiku 4.5 + Sonnet 4.6:

Library size	Pass-rate change
52 skills	−8 %
102 skills	−14 %
202 skills	−21 %

"Skill shadowing accounts for up to **68 %** of degradation at maximum library size, while **context overhead remains statistically indistinguishable from zero**."

Failure mode is model-dependent: Haiku *abandons* (selects no skill, −26 % at 202); Sonnet *mis-selects* (−15 %). Corroborated independently: tool-selection accuracy "drops from above 90 % with fewer than 30 candidates to 13.6 % with 11,100 options."

The local corpus is 5.8× past the worst point ever measured.

05_catalogue_survey.md counted **1174 SKILL.md** files in submodules/helix_agent/skills/. The study's worst case is 202. The curve is monotonically worsening at every sampled point.

Blast radius if unaddressed. "Fully incorporate... all power-features, nothing left out", read as *register every skill into one namespace*, is **precisely the configuration the literature says degrades the agent**. Both consumers get worse at their jobs while every unit, integration and gate test stays green — pass-rate degradation of this kind only shows up in end-to-end task success. It is a false-green of exactly the class §11.4 exists to prevent, and no amount of test coverage on the *loader* would catch it.

Critically, **the intuitive mitigations fix the wrong variable**. The instinctive worry is token cost; the measurement says token cost is statistically zero and the damage is **semantic interference between competing description fields**. Trimming prose, shortening SKILL.md bodies, or leaning on progressive disclosure does nothing — Level-1 descriptions are *always* loaded and *always* compete.

Mitigation (designed). Never mount the whole library. Four cooperating rules:

1. **Per-consumer opt-in subsets.** Each consumer declares the skill *sets* it activates; the default is none. Registration is an explicit allowlist, not a directory walk.
2. **Hard active-count ceiling**, enforced at load time — the loader refuses to activate more than N skills and names the offender. N is **calibrated on this project's own fixtures** (§11.4.6 — never borrowed from the paper's 202).
3. **Description-disambiguation gate.** Compute pairwise similarity across active description fields; fail on any pair above the calibrated threshold, since overlapping descriptions are the measured shadowing mechanism.
4. **Namespacing** (<source>.<name>) so growth in one source cannot retroactively shadow another (composes with R-04).

Proof plan. This risk is uniquely one where a gate is *not* sufficient — the harm is behavioural, so the proof must be behavioural. Build a **local SkillsBench analogue**: a fixed task set with known-correct skill selection, run at increasing active-library sizes, recording pass-rate per size. That curve is the calibration evidence for the ceiling in (2) and the threshold in (3), and it is re-run as a performance-type test (§11.4.169) on every corpus change. Gate CM-ACTIVE-SKILL-CEILING enforces the calibrated ceiling; paired §1.1 mutation: raise the ceiling past the measured knee → the benchmark test **MUST** show the regression. Evidence at `qa-results/hxc159/shadowing_curve/<run-id>/`.

Honest boundary (§11.4.6). The study measures Haiku 4.5 / Sonnet 4.6 on SkillsBench, not HelixCode on its own corpus. The *effect* is established; its *magnitude here* is not, and must be measured before any threshold is fixed. What is not in doubt is the direction, and that 1174 is far outside the measured range.

R-01 — The task's premise does not hold: HelixSkills ships nothing to incorporate

Class: EVIDENCED · **Severity:** CRITICAL

What it is. HXC-159 says "fully incorporate the HelixSkills System into HelixCode + HelixAgent — all power-features, nothing left out". Measured against HelixDevelopment/skills@315b56ce (complete recursive tree, truncated: false), there is no consumable artifact:

Probe	Result
.go files outside docs/	0
go.mod at root	absent (only docs/research/mvp/Agent_AI_Skill_Tree_Development/project/go.mod)
SKILL.md files	0
helix-deps.yaml at root	absent
tests/	4 shell scripts, all *constitution_inheritance* — none test the skills system
GitHub-detected language	Shell

Its own README (Revision 3) states it installs seven skills, all of which live at constitution/skills/<name>/ — and HelixCode already vendors that directory with exactly those seven entries.

Blast radius if unaddressed. An implementation plan written against the stated premise produces phases and tasks for a payload that does not exist. Work is declared done against a submodule pointer that adds no capability, while the real gap (no loader, no reconciliation, no format normalisation) is never scheduled. This is a §11.4 planning-layer bluff: green tasks, zero user-visible capability.

Mitigation (designed). Re-scope HXC-159 from *incorporate* to **build-then-incorporate**, in this order, and record the re-scope on the item itself so the premise change is visible rather than silently absorbed:

1. **Phase A — extend the source (§11.4.74 Verdict 1).** Promote the research MVP out of docs/research/.../project/ into a real root module in HelixDevelopment/skills; add a root helix-deps.yaml (CONST-054). Upstream this to the skills repo — never re-implement as a HelixCode-local helper.
2. **Phase B — normalise the corpus** (see R-05) so a loader has a well-defined input.
3. **Phase C — build the loader** (§11.4.74 Verdict 7 no-match) *inside* the skills repo.
4. **Phase D — incorporate** into both consumers, at the root layout R-02 requires.

An incorporation phase **MUST NOT** be scheduled before Phase C exists.

Proof plan. Pre-build gate CM-HXC159-SOURCE-CONSUMABLE: fails while HelixDevelopment/skills at the pinned SHA has no root go.mod **and** no helix-deps.yaml. Paired §1.1 mutation: point the gate at the pre-Phase-A SHA → gate **MUST** fail. Evidence: gate output at qa-results/hxc159/source_consumable/<run-id>/.

R-02 — Nested own-org submodule chain: adding skills violates CONST-051(C)

Class: EVIDENCED · **Severity:** CRITICAL

What it is. HelixDevelopment/skills has exactly one submodule, and it is constitution:

```
$ gh api repos/HelixDevelopment/skills/contents/.gitmodules |  
base64 -d  
[submodule "constitution"]  
  path = constitution  
  url = git@github.com:HelixDevelopment/HelixConstitution.git
```

HelixCode already carries constitution/ at its own root. Adding skills as a submodule therefore places the *same* own-org dependency at two paths under one checkout, pinned independently — precisely what CONST-051(C) / §11.4.28(C) forbid:

Nested own-org submodule chains are FORBIDDEN. Add the dependency at HelixCode's root; the consuming submodule reaches it via documented import/SDK/runtime resolver — never via its own nested `.gitmodules` entry.

Blast radius if unaddressed. Two constitution trees on disk; governance gates that glob `constitution/**` see two different rule sets; the §11.4.157 five-carrier cascade has two targets to keep in lockstep and will drift; and R-03's skew becomes structural rather than incidental. It is also a release blocker on its own terms.

Mitigation (designed). Root-level sibling layout with the nested chain removed:

```
<repo-root>/
├── constitution/          ← single source of truth, one pin
└── submodules/skills/    ← nested constitution/ NOT initialised
```

Concretely: add `skills` at `submodules/skills` per the grouped layout already used by the other 20+ entries in `.gitmodules`; do **not** recurse its `constitution` submodule (`git submodule update --init without --recursive` for that path, and an explicit `submodule.submodules/skills/constitution.update = none` entry); and have `skills` resolve its `constitution` through a documented **config-injected path** (env var, e.g. `HELIX_CONSTITUTION_DIR`, defaulting to `../../constitution`) rather than a nested checkout. The config-injection form is required anyway by §11.4.28(B) — `skills` must stay project-not-aware.

Upstream, the durable fix is to remove the nested pin from `skills` entirely and document the resolver, so no consumer has to special-case it.

Proof plan. Gate `CM-NO-NESTED-OWN-ORG-CHAIN`: walk every initialised submodule's `.gitmodules` for own-org URLs (`HelixDevelopment/*`, `vasic-digital/*`) that also appear at the root; any hit fails. Paired §1.1 mutation: initialise `submodules/skills/constitution` → gate **MUST** fail. Integration proof: `find . -name Constitution.md -path '*/constitution/*'` returns exactly one path. Evidence at `qa-results/hxc159/layout/<run-id>/`.

R-03 — Live 46-commit constitution pin skew, with measurable capability loss

Class: EVIDENCED · **Severity:** CRITICAL

What it is. The two pins already disagree, today, before any work:

```
HelixCode  constitution pin: ce3331a1  (2026-07-28 15:19:07 +0500)
= upstream tip
skills      constitution pin: 68875c7a  (2026-07-18 03:42:37 +0500)
```

```
$ git -C constitution log --oneline 68875c7a..ce3331a1 | wc -l
46
```

And the skew is **not cosmetic** — it changes the skills payload:

```
$ git -C constitution diff --stat 68875c7a..ce3331a1 -- skills/
  skills/reporting-workable-items/SKILL.docx | Bin 14458 -> 14458
bytes
  skills/reporting-workable-items/SKILL.pdf   | Bin 80490 -> 80489
bytes
  skills/workable-item-lifecycle/SKILL.docx   | Bin 0 -> 14102 bytes
  skills/workable-item-lifecycle/SKILL.html   | 496 ++++++
+++++
  skills/workable-item-lifecycle/SKILL.pdf    | Bin 0 -> 79245 bytes
  5 files changed, 496 insertions(+)
```

workable-item-lifecycle's rendered documents **do not exist at the pin HelixSkills carries**. A consumer resolving through HelixSkills gets a demonstrably smaller skill set than one resolving through HelixCode.

Blast radius if unaddressed. Two consumers silently expose different skill sets; a skill "works" in HelixCode and is absent in HelixAgent with no error anywhere; validation passes against whichever tree the test happened to resolve. This is the §11.4.108 SOURCE → RUNTIME gap at the dependency layer.

Mitigation (designed). R-02's single-root layout structurally removes the *second* pin — with one constitution/, skew becomes impossible by construction rather than by discipline. That is the rock-solid form and it is preferred.

Where a second pin cannot yet be removed (during the transition), add a **pin-equality gate** that compares git ls-tree HEAD constitution across every consumer and fails on inequality, plus a scheduled git submodule update --remote that advances all consumers together in one commit.

Proof plan. Gate CM-CONSTITUTION-PIN-SINGLE: assert exactly one constitution gitlink in the whole tree; if a transition period requires two, assert their SHAs are byte-equal. Paired §1.1 mutation: reset one

pin back to 68875c7a → gate MUST fail. Runtime proof (§11.4.108): the loader reports its resolved constitution SHA at startup, and a test asserts it equals `git ls-tree HEAD constitution`. Evidence at `qa-results/hxc159/pin_parity/<run-id>/`.

3. High risks

R-04 — Three disjoint skills corpora with zero reconciliation and zero name overlap

Class: EVIDENCED · **Severity:** HIGH

What it is. "Skills" is not one thing in this ecosystem:

Corpus	Location	SKILL.md
Constitution skills	constitution/skills/	3 (of 7 dirs)
HelixAgent skills	submodules/helix_agent/skills/ (tree 78ebdc67, 15 categories)	1174
"HelixSkills System"	HelixDevelopment/skills	0

`comm -12` over the sorted directory listings of corpora 1 and 2 returns **empty** — zero shared names. Nothing reconciles them.

The corpus with 1174 skills is the one *not* named in the task; the one named ships zero.

Blast radius if unaddressed. "Fully incorporate, nothing left out" is unsatisfiable without deciding which corpus is authoritative. Incorporating the named one adds a catalogue over a payload HelixCode already has, while 1174 skills stay invisible to HelixCode forever.

Mitigation (designed). Declare a **single authoritative registry** with typed sources rather than merging trees. In `HelixDevelopment/skills`, define a `sources`: manifest naming each corpus, its root, its format dialect, and its precedence; the loader enumerates all registered sources into one namespace and **refuses to start on a name collision**

(fail-closed, never last-writer-wins). Corpus 2 is registered in place — it is a plain tracked tree inside HelixAgent (R-18), so it is referenced, not copied.

Namespacing: qualify every skill as `<source>.<name>` so corpus growth can never retroactively collide.

Proof plan. Unit: collision fixture → loader returns a typed `ErrDuplicateSkill`, never silently resolves. Integration: enumerate all three sources; assert the union count equals the sum of per-source counts (no silent drops) — today that is $3 + 1174 + 0$. Paired §1.1 mutation: make the collision handler pick a winner instead of erroring → test MUST fail. Evidence at `qa-results/hxc159/registry_union/<run-id>/`.

R-05 — Skill filename case is split; one skill has no document at all

Class: EVIDENCED · **Severity:** HIGH

What it is. Measured across the seven constitution skills on this case-sensitive Linux filesystem:

```
HAS SKILL.md : action-prefix-system
NO  SKILL.md : media-validator      -> ships skill.md
(lowercase)
NO  SKILL.md : multitrack           -> ships register.sh ONLY –
no skill doc at all
HAS SKILL.md : reporting-workable-items
NO  SKILL.md : scheduled-work-queue -> ships skill.md
(lowercase)
NO  SKILL.md : session-sync         -> ships skill.md
(lowercase)
HAS SKILL.md : workable-item-lifecycle
```

A loader globbing `SKILL.md` finds 3 of 7. A loader globbing `skill.md` finds 3 *different* ones. `multitrack` is found by **neither**.

Upgraded in Rev 2 — this is not a style preference, it is non-conformance with a published standard. Agent Skills is now an open cross-vendor spec (<https://agentskills.io/specification>, accessed 2026-07-29) whose normative rules include: the canonical file is **SKILL.md**, and name **must match the parent directory name**. The measured corpus violates both. A conformance validator already exists

— `skills-ref validate ./my-skill` — so this is mechanically provable today, and conformance is what buys portability to Gemini CLI / Qwen Code / Codex, closing the §11.4.228 G1 gap.

Blast radius if unaddressed. A loader written against either convention silently loads under half the declared skills and reports success — the exact false-green class §11.4 forbids. It would also have been invisible on a case-insensitive filesystem (macOS), so it would ship and only break on Linux hosts and in containers.

Mitigation (designed). Normalise upstream in constitution, then enforce mechanically:

1. `git mv` each lowercase `skill.md` → `SKILL.md` (uppercase is the Anthropic convention and the majority already-correct form), updating every reference in the same commit per §11.4.29's reference-integrity rule.
2. Author the missing `multitrack/SKILL.md` — do not delete the skill (§11.4.124 investigate-before-remove; it has a `register.sh`, so it is wired, not dead).
3. Loader is **case-exact and fail-closed**: it globs `SKILL.md` only, and any directory containing a `register.sh` but no `SKILL.md` is a hard error, not a skip. A silent skip here is what makes the defect invisible.

Proof plan. Gate `CM-SKILL-MANIFEST-COMPLETE`: for every directory under each registered source root, assert `SKILL.md` exists with exact case (`test -f`, plus an explicit `ls | grep -x 'SKILL.md'` so a case-insensitive FS cannot fake a pass). Paired §1.1 mutation: rename one back to `skill.md` → gate **MUST** fail. Integration: loader enumerates all 7 constitution skills — asserting the count, not merely "no error". Evidence at `qa-results/hxc159/manifest_complete/<run-id>/`.

R-06 — A tracked, already-broken skill symlink points at a foreign-OS path

Class: EVIDENCED · **Severity:** HIGH

What it is. HelixCode's own `skills/` directory contains exactly one entry, and it is a broken symlink — **committed to git**, so every clone gets it:

```
$ git ls-files -s skills/
120000 7f326406b693365b6d1271b32acf477f38619863 0    skills/media-
```

validator

```
$ readlink skills/media-validator
/Volumes/T7/Projects/helix_code/constitution/skills/media-validator
```

```
$ readlink -e skills/media-validator
(no output – BROKEN: target unresolvable)
```

/Volumes/T7/... is a macOS mount path. It does not exist on this Linux host. The real directory is present at constitution/skills/media-validator, so the symlink is both broken *and* redundant. This is a concrete instance of the §11.4.228 G2 "symlinked skills" gap, and of §11.4.111 (bind by stable name, never by a host-specific absolute path).

Blast radius if unaddressed. Any loader that walks skills/ gets ENOENT on its only entry. Worse, a loader that treats a broken link as "absent" reports a clean empty scan — a false green. It also breaks the host-portability the project already depends on (this repo is worked on from both macOS and Linux).

Mitigation (designed). Replace the absolute symlink with a **repo-relative** one (../constitution/skills/media-validator) or, better, delete it and let the registry (R-04) reference constitution/skills/ as a registered source — no symlink at all. Absolute paths into /Volumes/**, /Users/**, /home/** are then banned by gate, because the same mistake will otherwise recur from the other OS.

Proof plan. Gate CM-NO-ABSOLUTE-OR-BROKEN-SYMLINKS: for every tracked symlink (git ls-files -s | awk '\$1==120000') assert (a) readlink -e resolves and (b) the target is not absolute. Paired §1.1 mutation: re-create the /Volumes/T7/... link → gate MUST fail. Evidence at qa-results/hxc159/symlink_hygiene/<run-id>/.

R-07 — Cross-consumer capability parity has no enforcing mechanism

Class: EVIDENCED (asymmetry measured) · **Severity:** HIGH

What it is. Two consumers, one library, and the two are already structurally asymmetric:

Property	HelixCode	HelixAgent
Go module		dev.helix.agent, go 1.26

Property	HelixCode	HelixAgent
	dev.helix.code, go 1.26	
constitution/ submodule	present , pinned ce3331a1	absent — no constitution entry in its 59-submodule .gitmodules
Own skills/ corpus	1 broken symlink	1174 SKILL.md
MCP seam	helix_code/ internal/mcp	(separate)

The Go versions match (1.26 / 1.26), which removes one skew axis. But HelixAgent has **no constitution submodule at all**, so a skills system that resolves skills *through* constitution/skills/ cannot resolve anything there.

Blast radius if unaddressed. A capability lands in one consumer and silently not the other — the exact failure the task flags. Because there is no parity gate, this is discovered by a user, not by a test.

Mitigation (designed). A **shared conformance suite owned by the library, run by both consumers**. The skills repo exports a capability manifest (the list of skills + the loader API surface it guarantees) plus an executable conformance test. Each consumer runs that suite in its own CI-equivalent local gate and publishes the resulting capability report. A parity gate diffs the two reports and fails on any asymmetry the manifest does not explicitly mark consumer-optional.

For HelixAgent specifically, the missing constitution/ must be resolved first — either add it at HelixAgent's root (same single-pin rule as R-02/ R-03) or make the constitution source path config-injected so HelixAgent supplies its own.

Proof plan. Gate CM-CROSS-CONSUMER-PARITY: run the library conformance suite in both consumers, diff the capability reports, fail on unexplained asymmetry. Paired §1.1 mutation: register a skill in one consumer only → gate MUST fail. Evidence: both reports + the diff at qa-results/hxc159/parity/<run-id>/.

R-08 — Test-surface explosion makes §11.4.169 unaffordable if scoped naively

Class: ANTICIPATED (from evidenced counts) · **Severity:** HIGH

What it is. §11.4.169 mandates 13 test types. R-04 measured 1174 + 3 + 0 skills. Naive scoping — every test type against every skill — is $13 \times 1177 \approx 15\,000$ test units, on top of stress/chaos/DDoS/memory/race suites that each need infrastructure. That plan is not affordable, and an unaffordable plan gets quietly skipped, which is how §11.4.169 turns into a bluff.

Blast radius if unaddressed. Either the suite is written and never runs to completion, or coverage is silently narrowed while still claiming the full matrix. Both are §11.4 violations at the coverage layer.

Mitigation (designed). Scope the matrix by **architectural surface, not by skill count** — the 13 types apply to the *loader and registry*, which is one component, while individual skills get a cheap uniform contract check:

Layer	Scope	Types applied
Loader / registry / resolver	1 component	all 13 (§11.4.169)
Per-skill	1177 skills	1 parameterised contract test (schema-valid, discoverable, non-colliding), table-driven — one test function, N cases
Representative skills	a sampled tier (~5) covering each source dialect	e2e + Challenge + HelixQA

This is genuine full coverage — every skill IS tested — while the expensive types land once on the component that can actually fail in those ways. A skill cannot deadlock; the loader can.

Proof plan. The coverage ledger (§11.4.25/§11.4.52) records feature × type × evidence-state and is itself gated: CM-MANDATORY-TEST-TYPES-COVERED fails if any of the 13 types is missing for the loader tier, or if

any registered skill lacks its contract-test case. Paired §1.1 mutation: drop the race-detector row → gate MUST fail. Evidence at qa-results/hxc159/coverage_ledger/<run-id>/.

4. Medium risks

R-09 — Host lacks X11/GL headers; any GUI-adjacent skill surface cannot build here

Class: EVIDENCED · **Severity:** MEDIUM

What it is. Verified on this host:

```
MISSING /usr/include/X11/Xlib.h
MISSING /usr/include/GL/gl.h
pkg-config gl: MISSING
```

HelixCode ships a Fyne desktop application (helix_code/applications/desktop/), which needs both. Any skill surface reaching into the desktop app cannot be built or tested on this host.

Mitigation (designed). Keep the skills loader **GUI-free by construction** — it is a data/registry component and must not import Fyne or any GL-linked package, so the entire skills test suite runs headless. Enforce that with a forbidden-import gate rather than convention. Where a genuinely GUI-bound skill surface is required later, it goes behind a build tag with a headless path, and if neither is possible it is an honest §11.4.3 SKIP-with-reason plus a tracked migration item — never a silent pass.

Proof plan. Gate CM-SKILLS-LOADER-HEADLESS: go list -deps on the loader package asserts no Fyne/GL/X11 dependency. Paired §1.1 mutation: add a Fyne import → gate MUST fail. Positive evidence: the full loader suite runs green on this header-less host — which is itself the proof. Evidence at qa-results/hxc159/headless/<run-id>/.

R-10 — §11.4.228's mandated extensions catalogue does not exist in HelixCode

Class: EVIDENCED · **Severity:** MEDIUM

What it is. §11.4.228 requires per-extension documentation with a compatibility matrix, and cites docs/extensions/EXTENSIONS_CATALOG.md as the live artifact. In this repo:

```
$ test -f docs/extensions/EXTENSIONS_CATALOG.md && echo PRESENT ||  
echo ABSENT  
ABSENT
```

The directory docs/extensions/ does not exist at all. Adding a skills system — which is by definition an extension mechanism — without this catalogue compounds an existing governance gap.

Mitigation (designed). Create docs/extensions/EXTENSIONS_CATALOG.md as part of Phase D, with the per-platform compatibility declaration §11.4.228 requires (which of Claude Code / OpenCode / HelixCode / Gemini CLI / Qwen Code loads each skill, and by what path), and register it in the docs-chain so it re-syncs automatically rather than by vigilance.

Proof plan. Gate CM-EXTENSIONS-CATALOG-PRESENT-AND-COMPLETE: file exists and every registered skill appears as a row with a non-empty per-platform column. Paired §1.1 mutation: register a skill without a catalogue row → gate MUST fail.

R-11 — Governance load per new submodule is real and easy to half-do

Class: EVIDENCED (requirements are in force; state measured) ·
Severity: MEDIUM

What it is. Adding skills triggers a fixed governance checklist, each item independently a release blocker:

Requirement	Anchor	Current state in skills@315b56ce
Five-carrier cascade	§11.4.157	CLAUDE/AGENTS/QWEN/ GEMINI present at root; no CONSTITUTION.md at root
helix-deps.yaml	§11.4.31 / CONST-054	absent at root (only inside the research MVP)
install_upstreams	§11.4.36 / CONST-056	upstreams/ present with 4 recipes (GitHub, GitLab, GitFlic, GitVerse) — must be run on add

Requirement	Anchor	Current state in skills@315b56ce
.gitignore + regeneration	§11.4.30 / §11.4.77	.gitignore present; regeneration metadata unverified

Mitigation (designed). A single **submodule-onboarding gate** that runs on any .gitmodules change and asserts the whole checklist at once, so partial onboarding is impossible. Add the missing root helix-deps.yaml and CONSTITUTION.md to skills upstream during Phase A (they are needed anyway for R-01), and invoke install_upstreams from the repo root as part of the add, verified by `git remote -v | grep -c push` matching the recipe count.

Proof plan. Gate CM-SUBMODULE-ONBOARDING-COMPLETE over every entry in .gitmodules. Paired §1.1 mutation: delete the new helix-deps.yaml → gate MUST fail. Evidence at qa-results/hxc159/onboarding/<run-id>/.

R-12 — .mcp.json hard-codes constitution skill paths, recoupling the consumer

Class: EVIDENCED · **Severity:** MEDIUM

What it is. The existing registration seam embeds a literal path:

```
"media-validator": {
  "command": "bash",
  "args": ["constitution/skills/media-validator/media-validator.sh"],
  "env": { "MEDIA_VALIDATOR_EVIDENCE_DIR": "qa-results/media-validator" }
}
```

Registration is per-skill and hand-maintained; there are 7 register.sh scripts (one per constitution skill) and **no central registrar** in constitution/scripts/. Every new skill means another hand-edited literal path.

Blast radius. Adding a source root (R-04) or moving one breaks every hard-coded entry at once, silently — MCP servers that fail to start are easy to not notice. It also violates §11.4.28(B): the path is consumer-specific knowledge baked into a place that should be generated.

Mitigation (designed). Generate `.mcp.json` skill entries from the registry manifest (R-04) instead of hand-authoring them, with source roots supplied by config injection. The generator is idempotent and its output is diff-checked in a gate, so drift between manifest and `.mcp.json` cannot persist.

Proof plan. Gate CM-MCP-ENTRIES-GENERATED: re-run the generator, diff against the committed `.mcp.json`, fail on any difference. Runtime proof (§11.4.108): each generated server actually starts and answers a probe — path correctness is proven by execution, not by the string looking right. Paired §1.1 mutation: hand-edit one path → gate MUST fail.

R-13 — Stale branches in the source repo invite integrating the wrong ref

Class: EVIDENCED · **Severity:** MEDIUM

What it is. `HelixDevelopment/skills` carries eight branches, including three identical stale agent worktrees:

<code>feature/catalog-docs</code>	<code>dd1edaa5</code>
<code>feature/deep-research</code>	<code>8c78272c</code>
<code>feature/testing-infra</code>	<code>6d7306db</code>
<code>helix_skills</code>	<code>06d77bd8</code>
<code>main</code>	<code>315b56ce</code>
<code>worktree-agent-a41ff2c21450bdf5c</code>	<code>25cb8ca0</code>
<code>worktree-agent-abb59d8974644d7a9</code>	<code>25cb8ca0</code>
<code>worktree-agent-adb9e192e0bc9275c</code>	<code>25cb8ca0</code>

`helix_skills` is a plausible-looking branch name that is **not** `main`.

Mitigation (designed). Pin the submodule to an explicit SHA on `main` (`315b56ce...`) with `branch =main` recorded in `.gitmodules`, never to a bare branch name. Reconcile `feature/*` branches into `main` per §11.4.181's merge-not-delete rule before Phase A, and retire the `worktree-agent-*` refs only after confirming `git log main..<ref>` is empty (§11.4.124 — no removal without proof).

Proof plan. Gate CM-SUBMODULE-PIN-ON-MAIN: assert the recorded gitlink SHA is an ancestor of the upstream `main` tip. Paired §1.1 mutation: repoint to `helix_skills` → gate MUST fail.

R-14 — Skill content is executable instruction: a malicious or buggy skill is a live attack surface

Class: EVIDENCED (upgraded in Rev 2 — research returned) · **Severity:** HIGH

What it is. Skills are instructions an agent follows, and several ship executable shell (media-validator.sh, session-sync.sh, 7 × register.sh). Registering 1174 skills of mixed provenance (R-04, corpus 2 spans vendor-specific trees like azure, github-copilot, openhands) means a large volume of agent-directing content enters the trust boundary at once.

Rev 2 — the research makes this concrete and worse than assumed. Three cited facts (full citations in 04_deep_research.md §A4):

1. **The ecosystem is already compromised.** Snyk "ToxicSkills" scanned 3,984 published skills: **36.82 % carried security flaws, 13.4 % critical, 36 % contained prompt injection, 76 confirmed malicious payloads** (8 still live at publication). The bar to publish was "a SKILL.md and a GitHub account that's one week old — **no code signing, no security review, no sandbox by default.**"
2. **allowed-tools is a GRANT, not a sandbox** — the most misread field in the format. Anthropic's own docs: it grants permission "**without prompting you for approval... It does not restrict which tools are available: every tool remains callable.**" A skill in a repo is a self-service privilege-escalation primitive.
3. **Dynamic context executes shell *before* the model sees the skill.** Datadog Security Labs: **!-prefixed commands "execute immediately (before Claude sees anything)".** Combined with (2): allowed-tools: Bash(*) plus a !-command is **unprompted arbitrary RCE at file-read time.**

And the pattern to design against is documented: postmark-mcp shipped **15 benign releases to build trust**, then added a data-exfiltration backdoor in v1.0.16. **Pinning a name is worthless; only pinning a content hash defends.**

Structurally: unlike VS Code (process isolation) or Zed (WASM sandbox + versioned interface), Agent Skills have **no isolation boundary of any kind** — and the open spec contains **no security section at all**, where MCP ships RFC-2119 MUST-level requirements.

Mitigation (designed). Trust tiering plus a fail-closed capability declaration:

1. Every registered source is tagged `first-party` / `vendored` / `third-party`.
2. Each skill declares required capabilities (filesystem, network, shell) in its front-matter; the loader enforces the declaration and **refuses** an undeclared capability rather than warning.
3. Skills that ship executable scripts are enumerated explicitly in the catalogue (R-10) so the executable surface is reviewable as a set, not per-file.
4. Provenance is pinned by **content hash** so a skill's text cannot change under a fixed pin — this is the specific defence against the `postmark-mcp` pattern.
5. **"disableSkillShellExecution": true** in managed settings, closing the dynamic-context RCE path (3) by default; any skill needing it is an explicit, reviewed exception.
6. **allowed-tools** and **!-dynamic-context** are **review-gated privileged constructs** — a skill declaring either cannot register without an explicit allowlist entry, because per (2) the field grants rather than restricts.
7. Corpus-2 skills (vendored, 1174 of them) default to the **vendored** tier and are **not activated** without opt-in — which composes with R-19's subsetting rule and means the security and shadowing mitigations reinforce each other.

Proof plan. Gate `CM-SKILL-CAPABILITY-DECLARED`: every registered skill has a capability block; any skill shipping `*.sh` without a declared `shell` capability fails. Security-type test (§11.4.169): a fixture skill attempting an undeclared network call **MUST** be refused — proving enforcement, not just declaration. Paired §1.1 mutation: make the enforcement warn instead of refuse → test **MUST** fail.

5. Bridge-integration risks (added Rev 2) and resolved items

R-20 — The middle of the SpecKit ↔ Superpowers Bridge is designed but NOT installed

Class: EVIDENCED · **Severity:** HIGH

What it is. The operator requires HelixSkills to compose with the Bridge. The Bridge *design* is real and substantial — 8 Markdown documents with full .html/.pdf/.docx siblings at constitution/docs/research/extensions/speckit_superpowers/implementation/ (EXTENSION_DEVELOPMENT.md alone is 92 KB). But probing the 7 layers on this host in this checkout, 2026-07-29:

Layer	Probe	Result
L2 Spec-Kit Core	specify --version	specify 0.13.5.dev0 ; .specify/ present; specs/ absent
L3 SuperSpec	command -v superspec	NOT INSTALLED
L4 SuperB	command -v superb	NOT INSTALLED
L5 SuperBridge MCP	grep .mcp.json	ABSENT
L6 Helix LLM	submodules/ helix_llm	present
L7 llama.cpp	dependencies/ LLama_CPP	present

Layers 3, 4 and 5 — orchestration, discipline enforcement, and execution — are absent. The ends of the pipeline exist; the middle does not.

Blast radius if unaddressed. HelixSkills would be wired into a **partially paper pipeline**. Any plan phase that says "route through SuperBridge" is unexecutable today, and a task claiming Bridge integration is complete would be a \$11.4.108 SOURCE-vs-RUNTIME bluff: the design is committed, the runtime is not there.

Mitigation (designed). Treat the Bridge as **two separable deliverables**: the *artifact discipline* (SpecKit spec/plan/tasks, L2 — installed and usable **now**) and the *execution pipeline* (L3–L5 — not installed). HelixSkills integrates against L2 immediately and declares L3–L5 an explicit dependency with an OPERATOR-BLOCKED-class gate rather than a silent assumption. Every Bridge-dependent phase carries a **layer-presence precondition** checked at runtime, which fails loudly rather than degrading silently.

Proof plan. Gate CM-BRIDGE-LAYER-PRESENCE: probe each of the 7 layers and emit a machine-readable presence report; any phase declaring a dependency on an absent layer fails. Paired §1.1 mutation: declare a dependency on L5 while it is absent → gate MUST fail. Evidence: the presence report at `qa-results/hxc159/bridge_layers/<run-id>/`.

Honest boundary (§11.4.6). Absence from PATH proves not-installed-on-this-host. It does **not** prove the projects do not exist — that is R-21.

R-21 — The Bridge's execution layer DOES NOT EXIST, and two of the three remaining layers are single-maintainer hobby projects

Class: EVIDENCED (research returned) · **Severity:** CRITICAL (upgraded from HIGH)

What it is. The Bridge design assigns layer 5 to "SuperBridge MCP (execution)". **There is no such project.**

```
$ api.github.com/search/repositories?q=superbridge+mcp
total_count: 0
```

npm search superbridge returns 20 results, **zero of them MCP servers** (an Electron IPC bridge, a crypto superchain SDK, a questdk plugin). The superpowers-bridge CHANGELOG contains no occurrence of "MCP", "daemon", or "SuperBridge". And the name is not merely absent — it was **explicitly rejected**: the real project's docs record *"SuperBridge hides the product's relationship to Superpowers"* as the reason for not using it, corroborated first-hand by the v1.5.0 changelog's naming-hierarchy entry.

Worse, **layer 4 refuses the role the design assigns it.** superpowers-bridge's README, verbatim: *"It owns no plan, task store, execution lifecycle, completion state, or convergence command."*

A 7-layer architecture whose execution tier is a non-existent MCP server is not an architecture — it is a diagram.

The remaining third-party layers, measured:

Dependency	Maintainers	Last push	Releases	Pinnable?	Risk
obra/ superpowers	large, 987 watchers	2026-07-28	v6.2.0, monthly+	Yes — official marketplace plugin	LOW

Dependency	Maintainers	Last push	Releases	Pinnable?	Risk
SuperB (RbBtSn0w/ spec-kit- extensions)	1 human + 2 own bots	2026-07-03	12 , immutable.zip	Yes	MEDIUM
SuperSpec (WangX0111/ superspec)	1 human, 10 lifetime commits	2026-06-02	2 — v1.0.0 was broken	Weak — mutable tag archive	HIGH
speckit- community catalog	1 human , personal account styled as an org (org API → 404 , 3 followers)	2026-07-28	n/a	n/a	MEDIUM
erophames/ superpowers- mcp (nearest real L5)	9 commits, no LICENSE	2026-03-15 (~4.5 mo dead)	none	No	HIGH — auto-git pulls remote instructions daily, unsandboxed

Two further facts that remove a false comfort:

- **SuperSpec's headline install command does not work** on a default SpecKit install — it is not in the first-party `catalog.json`; installation requires an explicit **mutable tag-archive URL** with no checksum or signature.
- **Catalog listing is not an audit.** SpecKit's own FAQ, verbatim: "*The Spec Kit maintainers do not review, audit, endorse, or support extension code.*" "It's in spec-kit's catalog" is a formatting check being misread as a security review.

Blast radius if unaddressed. Any HXC-159 phase that routes work "through SuperBridge" is unimplementable — the component does not exist, so the phase can never honestly close. Building on it would produce a plan whose central integration step is a §11.4 planning-layer bluff. Separately, installing SuperSpec by mutable archive URL, or superpowers-mcp with its daily unsandboxed instruction pull, would import exactly the R-14 supply-chain exposure into the execution path.

Mitigation (designed).

1. **Strike layer 5 from the plan.** It cannot be a dependency because it is not a thing. Either the execution role is filled by an **own-org component** (per §11.4.74 Verdict 2, HelixDevelopment/specifier is the candidate and must be probed first) or the Bridge is re-drawn honestly as the layers that exist. Amending the design is the correct action; papering over the gap is not.
2. **Tier the survivors by measured health.** obra/superpowers is the only dependency clearing an enterprise bar, and only because it ships through Anthropic's official marketplace — adopt it *via that channel*, plugin-namespaced. SuperB is acceptable pinned to an immutable release asset. **SuperSpec is not adoptable as-is:** require an org mirror (the caf-* pattern already mirrors ~60 CLI agents; vasic-digital/caf-spec-kit mirrors SpecKit itself) plus SHA pinning before any use.
3. **Never erophames/superpowers-mcp** in its current state — unlicensed alone disqualifies it, and the daily unpinned instruction pull is the live form of the R-14 attack.
4. **Record a bus-factor assessment per dependency** as a tracked artifact, not a one-time note, since three of four are under four months old.

Proof plan. Gate CM-BRIDGE-DEPS-REAL-AND-PINNED: every layer named in the plan resolves to an artifact that (a) exists, (b) has an org mirror or official-marketplace channel, (c) is pinned by SHA or immutable release asset, and (d) carries a recorded health assessment. Paired §1.1 mutation: re-introduce SuperBridge MCP as a named dependency → gate MUST fail on (a). Evidence at qa-results/hxc159/bridge_deps/<run-id>/.

R-22b — Namespace collision is narrower than feared, but the biting case is undocumented

Class: EVIDENCED · **Severity:** MEDIUM (refines R-22)

Good news, cited. Claude Code docs, verbatim: "**Plugin skills use a plugin-name:skill-name namespace, so they cannot conflict with other levels.**" Since obra/superpowers installs as a **plugin** and SuperB installs as spec-kit /speckit.superb.* commands, neither lands bare in .claude/skills/. Only SuperSpec's optional symlink path touches the personal namespace, claiming one distinctive name. Symlinks also de-duplicate: "*if the same target is reachable from more than one location, Claude Code loads the skill once.*"

Bad news, also cited. Claude Code **does not error and does not warn** on a skill name collision — it resolves silently by precedence, so the failure mode is *silent shadowing*, not a diagnostic. And anthropics/claude-code issue **#50486**, asking for plugin-name prefixing of plugin skills, was **CLOSED AS NOT PLANNED** with no maintainer reply. Related reports #25209 (project skills showing *both* instead of overriding — behaviour contradicting the documented rule) and #15065 remain open.

The case that actually bites here is undocumented: NO external solution found – original work (§11.4.8) for **two independent installers both writing a directory of the same name into one .claude/skills/ tree**. That collision is resolved at the *filesystem* level before Claude Code ever reads it — last writer wins, or the second install fails on EEXIST, depending on installer.

Mitigation. Exactly R-22's rules (1)–(3): reserved prefixes, pre-write collision check that aborts rather than overwrites, and a tracked manifest driving deterministic regeneration. Prefer **plugin packaging** for anything shared, since the vendor guarantees plugin namespaces cannot conflict — that is the one collision class solved upstream rather than by us.

Proof plan. As R-22, plus an install-order determinism test: run both registrars in both orders, assert byte-identical results (§11.4.50).

R-22 — .claude/skills/ namespace collision — now REAL, and silent

Class: EVIDENCED · **Severity:** HIGH

What it is. Speckit distributes its own commands **as skills**, as plain Markdown, into per-agent directories — .claude/skills for Claude Code. During *this session* a sibling agent ran `specify init`, and that directory in this repo now holds **exactly 10 speckit-* skills** created at 00:31:

```
speckit-analyze  speckit-checklist  speckit-clarify  speckit-constitution
speckit-converge speckit-implement  speckit-plan      speckit-specify
speckit-tasks    speckit-taskstoissues
```

Earlier in this same session `.claude/` held only `settings.json`, `settings.local.json` and `worktrees/`. **SpecKit now occupies the exact namespace a HelixSkills registrar would write into**, and the collision is observable, not hypothetical.

And it is invisible to git:

```
$ git check-ignore -v .claude/skills
.gitignore:133:.claude/*      .claude/skills
```

Blast radius if unaddressed. Two systems writing one ignored directory means (a) a name collision produces **no diff in git status** — it is silent; (b) the registered set is **host-local untracked state**, so a fresh clone has zero skills and "the skills are installed" is unreproducible from the repository (a §11.4.77 regeneration gap); and (c) whichever registrar runs last wins, non-deterministically (§11.4.50).

Mitigation (designed). Three rules, all fail-closed:

1. **Disjoint prefixes by construction.** SpecKit owns `speckit-*`; HelixSkills registers only under its own reserved prefix. The registrar **refuses** to write any name outside its prefix.
2. **Pre-write collision check.** The registrar enumerates the target directory and aborts on any existing name it did not itself write — never overwrites.
3. **Declarative manifest + regeneration.** A tracked manifest lists what *should* be registered; a `scripts/register_skills.sh` (see R-24) regenerates the ignored directory from it deterministically, making the host-local state reproducible and satisfying §11.4.77.

Proof plan. Gate CM-SKILL-NAMESPACE-DISJOINT: enumerate `.claude/skills/`, assert every entry maps to exactly one owning registrar via prefix, and assert the set equals the manifest. Paired §1.1 mutation: plant a colliding `speckit-plan` under the HelixSkills prefix → gate MUST fail. Integration proof: run both registrars in both orders and assert byte-identical results (§11.4.50). Evidence at `qa-results/hxc159/namespace/<run-id>/`.

R-23 — A third constitution now exists in this repo (CONST-059 canonical-root ambiguity)

Class: EVIDENCED · **Severity:** MEDIUM

What it is. `specify init` created `.specify/memory/constitution.md`.
Measured:

Layer	Path	Size
Canonical root	constitution/Constitution.md	1,513,995 bytes
Consumer extension	CONSTITUTION.md	352,111 bytes
SpecKit (new)	.specify/memory/constitution.md	2,346 bytes

None declares inheritance from another, and /speckit.constitution will rewrite the third without reference to the first two. There is no documented mechanism to point SpecKit at an existing external constitution.

A correction I owe (§11.4.6). The widely-repeated claim that SpecKit *imposes* a nine-article constitution with a "three projects maximum" cap is **not true of the installed artifact**. What `specify init` actually writes is a **placeholder template** (`# [PROJECT_NAME] Constitution, [PRINCIPLE_1_NAME], ...`); the nine articles appear only as **HTML-comment examples**. The doctrinal risk is therefore much milder than the documentation implies. Reporting it as prescribed would have been a bluff, and the risk that remains is **structural** (a third un-inherited constitution file), not doctrinal.

Mitigation (designed). Fill `.specify/memory/constitution.md` with an explicit **inheritance pointer** to the canonical root — the CONST-059 pattern already used by `CLAUDE.md` — so it is unambiguously a consumer extension and never a competing source of truth. Add it to the §11.4.157 lockstep set so it cannot drift.

Proof plan. Gate CM-CANONICAL-ROOT-CLARITY extended to `.specify/memory/`: assert the file opens with the inheritance pointer. Paired §1.1 mutation: strip the pointer → gate MUST fail.

R-24 — §11.4.164 auto-propagation is only half-wired: the consumer registrars do not exist

Class: EVIDENCED · **Severity:** MEDIUM

What it is. §11.4.164 specifies that a constitution pull triggers `post_update_hook.sh`, which registers new/modified skills and MCP servers via the *consumer's* registrars. The hook exists; the registrars do not:

```
$ ls -la constitution/scripts/post_update_hook.sh
-rwxr-xr-x 20975 bytes  ← EXISTS
```

```
$ ls scripts/register_skills.sh scripts/register_mcp.sh
ls: cannot access 'scripts/register_skills.sh': No such file or
directory
ls: cannot access 'scripts/register_mcp.sh': No such file or
directory
```

So on a constitution pull, HelixCode has **no consumer-side implementation to invoke**. This is a pre-existing §11.4.164 gap that HXC-159 inherits — and it is the same missing component R-22's mitigation (3) requires.

Mitigation (designed). Author scripts/register_skills.sh and scripts/register_mcp.sh as the manifest-driven, idempotent, prefix-scoped registrars R-22 specifies. One component closes both gaps.

Proof plan. Gate CM-CONSTITUTION-AUTO-PROPAGATION (§11.4.164's own gate): hook present, registrars present and executable, and a dry-run registers a fixture skill. Paired §1.1 mutation: add a skill to constitution/skills/ and assert the hook detects and registers it; remove the registrar → gate MUST fail.

R-25 — replace directives are ignored outside the main module (10 already in play)

Class: EVIDENCED · **Severity:** MEDIUM (resolves former R-17)

What it is. Go Modules Reference, verbatim: "replace directives **only apply in the main module's go.mod file and are ignored in other modules**." Same for exclude. There is no lock file.

This is **the mechanism** behind "a capability lands in one consumer and not the other", and it is already live here — submodules/helix_agent/go.mod carries **at least 10** relative-path replaces:

```
replace dev.helix.dag          => ../dag_orchestrator
replace digital.vasic.containers => ../containers
replace digital.vasic.challenges => ../challenges
replace digital.vasic.agentic   => ../agentic
replace digital.vasic.llmops    => ../llm_ops
replace digital.vasic.selfimprove => ../self_improve
replace digital.vasic.planning  => ../planning
replace digital.vasic.benchmark => ../benchmark
```

```
replace digital.vasic.llmsverifier => ../llms_verifier/llm-  
verifier
```

Every one resolves **only** when `helix_agent` is the main module. And `go.work` does not save this: workspaces are local-development tools that CI must not use, so local-green and release-real diverge — a textbook §11.4.108 SOURCE → ARTIFACT gap.

Mitigation (designed). Keep the shared skills library **dependency-free of replace-requiring modules** — which the pure-data shape (R-15, resolved) makes easy, since a markdown corpus has no Go dependencies at all. Where any Go code is shared, add a **version-parity gate** asserting every consumer's `require` on the shared library is byte-identical, plus the registry-completeness assertion from R-15 that fails the *consumer's* startup when an expected capability is absent.

Proof plan. Gate CM-CONSUMER-VERSION-PARITY: extract the shared-library `require` line from each consumer's `go.mod`; fail on inequality. Paired §1.1 mutation: bump one consumer only → gate MUST fail.

R-26 — Root and inner modules declare the SAME module path

Class: EVIDENCED (flagged unverified by research; verified by me) ·
Severity: MEDIUM

What it is.

```
go.mod:           module dev.helix.code    (go 1.25.2)  
helix_code/go.mod: module dev.helix.code    (go 1.26)
```

Two modules sharing one module path, on different Go versions, in one repository. This is not the diamond problem of R-25 but is its own resolution hazard, and it sits directly under any shared-library seam.

Mitigation (designed). Do not build the skills seam on top of this ambiguity. Either rename the thin root module to a distinct path (e.g. `dev.helix.code/meta`) or ensure the skills library is consumed **only** by the inner module, never the root. Decide explicitly and record it — do not let resolution order decide.

Proof plan. Gate CM-UNIQUE-MODULE-PATHS: collect every `module` line in the repo; fail on duplicates. Paired §1.1 mutation: re-introduce the duplicate → gate MUST fail.

R-27 — `/speckit.taskstoissues` forks the workable-items single source of truth

Class: EVIDENCED · **Severity:** MEDIUM

What it is. Speckit ships `/speckit.taskstoissues`, which pushes tasks into **GitHub Issues**. HelixCode's §11.4.93/§11.4.95 mandate a git-tracked SQLite DB (`docs/workable_items.db`) as the single source of truth for workable items. Both would hold task state, neither is authoritative over the other, and the skill is registered and invocable **right now** (R-22 lists it).

Mitigation (designed). Disable or re-target the command: either remove `speckit-taskstoissues` from the registered set (the R-22 manifest makes this a one-line declarative exclusion) or re-point it at the DB via the existing `workable-items` tooling. The DB stays authoritative; GitHub Issues, if used at all, is a derived view per §11.4.148's tracker-sync model.

Proof plan. Gate CM-WORKABLE-ITEMS-SSOT-UNFORKED: assert `speckit-taskstoissues` is absent from the registered manifest, or that its target is the DB. Paired §1.1 mutation: re-register it unmodified → gate MUST fail.

R-28 — Corpus 2 is not reusable as-is (formerly R-18)

Class: EVIDENCED · **Severity:** MEDIUM — needs an explicit decision, not a default

`submodules/helix_agent/skills/` is a plain tracked tree (78ebdc67), not a submodule, so HelixCode cannot consume it without either lifting it into its own repo (invasive — and §11.4.122 no-silent-removal applies to anything taken out of HelixAgent) or referencing it through HelixAgent (couples the two consumers, which §11.4.28 forbids).

Mitigation (designed). Extract the corpus into its own own-org repo at the consumer root (CONST-051(C) layout), leaving HelixAgent consuming it as a registered source rather than owning it. Because this **removes** a tree from HelixAgent, it is a §11.4.122 event: the operator must be asked before, not told after. Until that decision, corpus 2 stays vendored-tier and unactivated (R-14 mitigation 7), which is also what R-19 requires.

Proof plan. Gate CM-SKILL-SOURCE-AT-ROOT: every registered source resolves to a root-level path, never through another consumer. Paired §1.1 mutation: register a source via submodules/helix_agent/skills → gate MUST fail.

Resolved from Rev 1

- **R-15 — Go extension mechanism. RESOLVED → pure-data skills.** plugin is eliminated by its own documentation: "supported only on Linux, FreeBSD, and macOS, making them unsuitable for applications intended to be portable" (HelixCode's make prod cross-compiles Windows) and — decisively — **"The application and its plugins must all be built together by a single person or component of a system"**, which is the negation of the two-independent-consumers requirement. SpecKit is the working existence proof of the alternative at 50-agent scale: skills are plain Markdown copied into per-agent directories, no ABI, no linker. Fallback ranking if code loading is ever genuinely needed: compile-time registry → wazero/WASM (only option with true sandboxing and Windows, no CGO) → MCP-over-stdio → hashicorp/go-plugin.
- **R-16 — SpecKit constitution collision. RESOLVED, and milder than feared** — see R-23. The installed artifact is a placeholder template, not a prescriptive nine-article document. Residual risk is structural (third un-inherited file), and brownfield adoption is empirically fine (spec-kit discussion #1119 reports minimal friction across a five-repo workspace).
- **R-17 — Go diamond dependency. RESOLVED → R-25.** The real mechanism is replace/exclude being ignored outside the main module, not MVS version selection.
- **R-18 — renumbered to R-28** for consistency with the Rev 2 ordering.

6. Summary table

ID	Risk	Class	Severity
R-19	Skill shadowing — literal execution makes both agents measurably worse	EVIDENCED	CRITICAL
R-01	HelixSkills ships nothing consumable — task premise fails	EVIDENCED	CRITICAL
R-02		EVIDENCED	CRITICAL

ID	Risk	Class	Severity
	Nested own-org submodule chain violates CONST-051(C)		
R-03	46-commit constitution pin skew with capability loss	EVIDENCED	CRITICAL
R-04	Three disjoint skills corpora, zero reconciliation	EVIDENCED	HIGH
R-05	Case split + multitrack undocumented = spec non-conformance	EVIDENCED	HIGH
R-06	Tracked broken symlink to a macOS path	EVIDENCED	HIGH
R-07	No cross-consumer parity mechanism; consumers asymmetric	EVIDENCED	HIGH
R-08	§11.4.169 test-surface explosion (~15 000 units if naive)	ANTICIPATED	HIGH
R-14	Skill content as attack surface (36.8 % of published skills flawed)	EVIDENCED	HIGH
R-21	Bridge execution layer (SuperBridge MCP) DOES NOT EXIST	EVIDENCED	CRITICAL
R-20	Bridge layers L3/L4/L5 designed but NOT installed	EVIDENCED	HIGH
R-22	.claude/skills/ namespace collision — real, silent, gitignored	EVIDENCED	HIGH
R-22b	Collision narrower than feared; biting case undocumented	EVIDENCED	MEDIUM
R-09	X11/GL headers missing on host	EVIDENCED	MEDIUM
R-10	§11.4.228 extensions catalogue absent	EVIDENCED	MEDIUM
R-11	Submodule governance checklist easy to half-complete	EVIDENCED	MEDIUM
R-12	.mcp.json hard-codes skill paths; no central registrar	EVIDENCED	MEDIUM
R-13	Stale/decoy branches in source repo	EVIDENCED	MEDIUM

ID	Risk	Class	Severity
R-23	Third constitution — CONST-059 canonical-root ambiguity	EVIDENCED	MEDIUM
R-24	§11.4.164 half-wired — consumer registrars absent	EVIDENCED	MEDIUM
R-25	replace ignored outside main module (10 already in play)	EVIDENCED	MEDIUM
R-26	Root + inner modules share the path <code>dev.helix.code</code>	EVIDENCED	MEDIUM
R-27	<code>/speckit.taskstoissues</code> forks the workable-items SSoT	EVIDENCED	MEDIUM
R-28	HelixAgent corpus not reusable as-is (§11.4.122 applies)	EVIDENCED	MEDIUM
R-15	Go extension mechanism → RESOLVED: pure-data skills	RESOLVED	—
R-16	SpecKit constitution collision → RESOLVED, milder (→ R-23)	RESOLVED	—
R-17	Go diamond dependency → RESOLVED (→ R-25)	RESOLVED	—
R-18	→ renumbered R-28	—	—

Counts: **29 risks**, of which **27 EVIDENCED**, **2 ANTICIPATED** (R-08 and the calibration half of R-19), **5 CRITICAL**, **3 RESOLVED** during Rev 2.

The three most dangerous, in order:

1. **R-19 (skill shadowing)** — because it is the only risk here that damages the product while every gate stays green, and because the corpus is **5.8× past the worst point ever measured**. Executing the task as literally worded — register every skill into one namespace — is the exact configuration the literature says degrades the agent. Mitigation: per-consumer opt-in subsets + a calibrated active ceiling + a description-similarity gate, proven by a **local benchmark curve**, not by a static gate, because the harm is behavioural.
2. **R-21 (the execution layer does not exist)** — because a plan phase that routes work through a non-existent component can never honestly close, and because it was believable enough to survive into a design document with 8 files and full four-format exports. Mitigation: strike layer 5, probe the own-org candidate (`HelixDevelopment/specifier`) first, and tier the surviving dependencies by measured health — never adopt SuperSpec by mutable archive URL or `superpowers-mcp` at all.

3. **R-01 + R-02 + R-03 (empty premise, nested chain, 46-commit skew)** — one cluster, because they share a fix. The premise gap converts the whole plan into work against an absent payload; the nested chain and pin skew are a single structural defect with observable capability loss *today*. Mitigation: re-scope to build-then-incorporate (no incorporation phase schedulable before the loader exists) on a single root constitution/ with no nested pin — making the skew impossible by construction rather than by discipline.

The pattern across all three: each was invisible to the task as stated and became visible only by measuring the artifact rather than reading the description. That is the §11.4.6 discipline doing its job, and it is why this register is a planning deliverable rather than a post-hoc note.

7. Honest boundary

Per §11.4.6, this register enumerates risks **detectable in advance from static inspection and catalogue survey**. It does not and cannot prove the set is complete — §11.4.118 is explicit that absence of evidence of looking is not evidence of absence. What is earned here is: "the enumerated inspection surface below was exercised, and these are its findings."

Inspection surface actually exercised: HelixDevelopment/skills complete recursive tree + branches + README + .gitmodules; HelixDevelopment/specifier root tree; both GitHub org listings; the GitLab vasic-digital listing; HelixCode's local constitution/, skills/, .claude/skills/, .specify/, .mcp.json, go.mod, helix_code/go.mod, helix_code/internal/{mcp, hooks}; HelixAgent's go.mod (incl. replace directives), .gitmodules, skills/ tree; the Bridge design corpus; the 7-layer runtime presence probe; host header availability. Plus four independent research angles against live external sources (04_deep_research.md).

Not exercised (honest gaps, each a tracked follow-up):

- the GitLab HelixDevelopment group — query-shape failure, see 05_catalogue_survey.md §1;
- the **runtime behaviour of any skill** — nothing was executed;
- **specifier as the layer-5 replacement candidate** (R-21 mitigation 1) — named but unprobed; its actual capability is unverified;
- specifier's actual capability beyond its root tree;

- the contents of the 1174 HelixAgent skills beyond count and category names;
- the four remaining *skills* branches' diffs;
- the **magnitude of R-19's shadowing effect on this project's own corpus** — the effect is established in the literature, the local magnitude is not, and must be measured before any threshold is chosen.

Two claims in this register were **corrected against measurement rather than inherited from documentation**, and both corrections mattered: Speckit's installed constitution is a placeholder template, not a prescriptive nine-article document (R-23); and the 10 registered speckit skills initially found on this host belonged to a *different project* before *specify init* created HelixCode's own set during this session (R-22). Both would have been plausible-sounding bluffs.

No mitigation in this register has been implemented or proven yet — each carries a *designed* mitigation and a *planned* proof. Per §11.4.108 none of them is "done" until its runtime signature verifies on a clean target.

No mitigation in this register has been implemented or proven yet — each carries a *designed* mitigation and a *planned* proof. Per §11.4.108 none of them is "done" until its runtime signature verifies on a clean target.